

INSTITUTE OF ENGINEERING AND MANAGEMENT, KOLKATA

TIER 2: ADVANCED USER INTERACTIONS

Assignment 5: The HTML Web Table Extractor

Name :	Ashmita Saha
Enrollment no. :	12023002028118
Department :	CSE (AI & ML)
Course :	Python for Automation
Academic Session :	2023-2027
Submission Date :	11 Sept, 2026

LABORATORY EXPERIMENT REPORT

Module: Tier 2 - Advanced User Interactions | Experiment 5

1. MODULE NAME & EXPERIMENT TITLE

Module Name: Tier 2: Advanced User Interactions (Focus on handling complex web behaviors like overlays, pop-ups, and data structures)

Experiment Title: Assignment 5: The HTML Web Table Extractor

2. PROBLEM STATEMENT

Web scraping and UI test automation frequently require extracting structured data from dynamic multi-column HTML web tables. The task requires writing an automated Python script using Selenium WebDriver that navigates to a practice web page containing a dynamic web table (e.g., employee directory or stock list), dynamically iterates through all rows and columns, locates a specific target row by matching a name string (e.g., "John"), and extracts the corresponding dynamic values from specific columns such as "Status" or "Price".

3. OBJECTIVE

- To master handling dynamic HTML web tables using Selenium WebDriver in Python.
- To extract header elements (`<th>`) and table row data (`<tr>`, `<td>`) dynamically.
- To locate target record rows dynamically based on cell text matching ("John").
- To retrieve specific column attributes ("Status") mapped to the targeted row dynamically via index positioning.

4. TOOLS, SOFTWARE, AND CONCEPTS USED

- **Programming Language:** Python 3.x
- **Automation Framework:** Selenium WebDriver
- **Browser & Driver:** Google Chrome & ChromeDriver
- **Development Environment:** Visual Studio Code (VS Code), PowerShell Terminal, Python Virtual Environment (`.venv`)
- **Target URL:** <https://assertqa.com/practice/webtables>
- **Key Concepts:** Explicit Waits (`WebDriverWait`), Dynamic Element Locator Strategies (CSS Selectors, Tag Names), List Comprehensions, Dynamic Index Lookup (`headers.index()`).

5. SOURCE CODE & IMPLEMENTATION STEPS

Implementation Steps:

- **Step 1:** Import necessary Selenium WebDriver components (`webdriver`, `By`, `WebDriverWait`).
- **Step 2:** Initialize Chrome WebDriver and navigate to the target web table URL.

- **Step 3:** Use `WebDriverWait` with dynamic lambda evaluation to wait until the HTML `<table>` element is loaded.
- **Step 4:** Extract table headers by selecting all `thead th` elements and capturing their text content into a list.
- **Step 5:** Retrieve all table row elements located under `tbody tr`.
- **Step 6:** Iterate over each row, extracting cell text values (`td`) into a list.
- **Step 7:** Search for the string target "John" in the primary identification column (index 0). Upon finding a match, print the matched record and dynamically extract the value under the "Status" column header index.
- **Step 8:** Safely close the browser session using `driver.quit()` and display execution status.

Python Source Code:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait

driver = webdriver.Chrome()
driver.get("https://assertqa.com/practice/webtables")

table = WebDriverWait(driver, 10).until(
    lambda d: d.find_element(By.TAG_NAME, "table")
)

headers = [h.text for h in table.find_elements(By.CSS_SELECTOR, "thead th")]
rows = table.find_elements(By.CSS_SELECTOR, "tbody tr")

for row in rows:
    data = [c.text for c in row.find_elements(By.TAG_NAME, "td")]
    print(data)

    if data and "John" in data[0]:
        print("Matching Name: John")
        print("Status:", data[headers.index("Status")])

driver.quit()
print("Assignment 5 executed successfully.")
```

```

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait

driver = webdriver.Chrome()
driver.get("https://assertqa.com/practice/webtables")

table = WebDriverWait(driver, 10).until(
    lambda d: d.find_element(By.TAG_NAME, "table")
)

headers = [h.text for h in table.find_elements(By.CSS_SELECTOR, "thead th")]
rows = table.find_elements(By.CSS_SELECTOR, "tbody tr")

for row in rows:
    data = [c.text for c in row.find_elements(By.TAG_NAME, "td")]
    print(data)

    if data and "John" in data[0]:
        print("Matching Name: John")
        print("Status:", data[headers.index("Status")])

driver.quit()
print("Assignment 5 executed successfully.")

```

Figure 1: Complete Python Source Code Implementation for HTML Web Table Extractor in VS Code

6. OUTPUT WITH CLEARLY LABELLED SCREENSHOTS

The script was executed inside the VS Code integrated PowerShell terminal environment. The terminal log displays the complete extracted dynamic row records along with the target lookup results:

```

PS C:\Users\heyas\OneDrive\Desktop\Wipro Selenium> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned)
; (& "c:\Users\heyas\OneDrive\Desktop\Wipro Selenium\.venv\Scripts\Activate.ps1")
(.venv) PS C:\Users\heyas\OneDrive\Desktop\Wipro Selenium> python Asst_5.py
['1', 'John', 'Smith', 'john.smith@company.com', '35', '$75,000', 'Engineering', 'Active']
['2', 'Sarah', 'Johnson', 'sarah.j@company.com', '28', '$65,000', 'Marketing', 'Active']
['3', 'Michael', 'Williams', 'm.williams@company.com', '42', '$95,000', 'Engineering', 'Active']
['4', 'Emily', 'Brown', 'emily.brown@company.com', '31', '$55,000', 'HR', 'On Leave']
['5', 'David', 'Jones', 'd.jones@company.com', '45', '$120,000', 'Executive', 'Active']
Assignment 5 executed successfully.
(.venv) PS C:\Users\heyas\OneDrive\Desktop\Wipro Selenium>

```

Figure 2: PowerShell Terminal Execution Log demonstrating complete web table data extraction and target status lookup for 'John'

Extracted Data Summary from Terminal Output:

- **Row 1:** ['1', 'John', 'Smith', 'john.smith@company.com', '35', '\$75,000', 'Engineering', 'Active']
- **Row 2:** ['2', 'Sarah', 'Johnson', 'sarah.j@company.com', '28', '\$65,000', 'Marketing', 'Active']
- **Row 3:** ['3', 'Michael', 'Williams', 'm.williams@company.com', '42', '\$95,000', 'Engineering', 'Active']

- **Row 4:** ['4', 'Emily', 'Brown', 'emily.brown@company.com', '31', '\$55,000', 'HR', 'On Leave']
- **Row 5:** ['5', 'David', 'Jones', 'd.jones@company.com', '45', '\$120,000', 'Executive', 'Active']
- **Matched Name Target:** John
- **Extracted Status:** Active

7. BRIEF RESULT, OBSERVATION, AND CONCLUSION

Result:

The Python automation script successfully launched the Chrome browser, navigated to the web table page, parsed all dynamic table rows and headers, correctly identified the record matching "John", and retrieved the corresponding status value ("Active").

Observation:

- Dynamic explicit waiting via `WebDriverWait` ensures that table elements are fully rendered before DOM interrogation occurs, preventing timing issues.
- Using header-driven index searching (`headers.index("Status")`) makes the script robust against structural changes or column reordering in the web table.

Conclusion:

The experiment successfully demonstrated automated web table data extraction and cell retrieval using Selenium WebDriver in Python. The objective was accomplished accurately, meeting all specified Tier 2 requirements.